Resenha: The Hexagonal (Ports & Adapters) Architecture (Cockburn)

Isolamento do núcleo da aplicação por portas, adaptadores e dependências tecnológicas substituíveis

Referência

COCKBURN, Alistair. **The Hexagonal (Ports & Adapters) Architecture**. HaT Technical Report 2005.02, versão 0.9, 4 set. 2005.

Fonte 1. Referências conferidas em 10 out. 2026.

1. Introdução

O artigo de Alistair Cockburn discute um problema recorrente no desenvolvimento de software: a mistura entre regras de negócio e detalhes de interface, banco de dados e integrações externas. Quando a aplicação passa a depender diretamente dessas tecnologias, o sistema fica mais difícil de testar, mais rígido para evoluir e mais sensível a mudanças no ambiente. A tese central do texto é que a aplicação deve ser desenhada a partir de uma separação nítida entre o que está dentro do núcleo de negócio e o que está fora dele. Para isso, Cockburn propõe a arquitetura hexagonal, também chamada de ports and adapters, em que o núcleo conversa com o exterior por contratos estáveis, sem conhecer diretamente a tecnologia concreta usada em cada lado.

2. Síntese das ideias principais

O texto é organizado de forma bastante didática. Primeiro, o autor apresenta a motivação do padrão, mostrando os prejuízos causados pelo acoplamento da lógica de negócio à interface e à infraestrutura. Depois, explica a solução conceitual baseada na assimetria entre interior e exterior da aplicação, detalha o papel das portas e dos adaptadores, ilustra a proposta com diagramas e um exemplo de código, e encerra com observações práticas sobre tipos de adaptadores, escrita de casos de uso e quantidade adequada de portas.

2.1 Separação entre núcleo de negócio e detalhes externos

A primeira ideia forte do artigo é que o verdadeiro erro arquitetural não está em escolher interface gráfica, banco relacional ou integração HTTP, mas em deixar que essas decisões contaminem o núcleo da aplicação. Cockburn mostra que, quando a regra de negócio se infiltra na camada de interface, o sistema perde testabilidade e passa a depender de detalhes voláteis, como telas, botões e fluxos visuais. O mesmo ocorre do outro lado, quando a lógica central fica presa à presença de um banco ou serviço específico. A arquitetura hexagonal nasce justamente para impedir esse vazamento.

2.2 Portas como contratos e adaptadores como tradução tecnológica

A noção de porta é apresentada como um contrato de conversa orientado por propósito. A aplicação expõe ou consome capacidades por meio dessas portas, enquanto os adaptadores fazem a tradução entre esse contrato e a tecnologia real utilizada. Assim, uma interface web, um script batch, um teste automatizado ou outra aplicação podem acionar a mesma porta de entrada sem alterar a regra de negócio. Do mesmo modo, banco SQL, arquivo, serviço remoto ou repositório em memória podem ocupar a mesma porta de saída. O ganho conceitual é claro: o núcleo passa a depender de abstrações de uso, não de implementações técnicas.

2.3 Testabilidade, execução headless e substituição de dependências

Um dos méritos mais práticos do artigo é mostrar que a arquitetura hexagonal melhora o ciclo de desenvolvimento, e não apenas o desenho abstrato do sistema. Como o núcleo pode ser acionado por testes e trabalhar com adaptadores substituíveis, torna-se possível validar comportamento sem interface gráfica e sem banco real. Isso reduz custo de regressão, acelera integração contínua e permite que equipes desenvolvam partes do sistema mesmo quando dependências externas estão indisponíveis. O exemplo com mock repository deixa evidente que a substituição de infraestrutura não é um detalhe secundário, mas parte da estratégia arquitetural.

2.4 Assimetria prática entre adaptadores primários e secundários

Embora o texto insista corretamente que a assimetria principal é entre dentro e fora da aplicação, Cockburn também reconhece uma diferença prática entre os lados: alguns adaptadores dirigem o sistema, enquanto outros são dirigidos por ele. Daí surgem os adaptadores primários e secundários, equivalentes aos atores primários e secundários dos casos de uso. Essa distinção é útil porque esclarece por que frameworks de teste costumam aparecer como entrada e mocks de banco como saída. Além disso, o autor defende que os casos de uso sejam escritos no limite da aplicação, e não contaminados por detalhes da tecnologia externa.

2.5 Implicações arquiteturais e trade-offs

O artigo não vende a ideia como uma fórmula mecânica. Ele admite que a definição da quantidade de portas depende do problema e que não existe um número universalmente correto. O ponto importante é organizar as interfaces por propósito, e não por conveniência de tecnologia ou pelo desenho clássico em camadas. Em troca, a solução introduz um nível adicional de abstração e exige disciplina no desenho dos contratos, para que as portas não virem duplicação artificial nem os adaptadores se encham de lógica de negócio. O trade-off, portanto, é trocar acoplamento acidental por uma estrutura mais explícita e sustentável.

3. Aplicação imaginada dos conceitos

Em um sistema de gestão de ordens de serviço, por exemplo, o núcleo da aplicação poderia concentrar regras como abertura de chamado, cálculo de prioridade, alocação de técnico e transição de status. Esse núcleo seria acessado por portas de entrada usadas tanto por uma interface web quanto por um aplicativo mobile e por um processo automatizado de importação de demandas vindas de parceiros. No lado de saída, as portas poderiam atender persistência, envio de notificações e integração com APIs externas, cada uma com adaptadores reais e versões de teste em memória.

Esse desenho permitiria validar fluxos críticos sem depender de navegador, banco ou API de terceiros, além de facilitar troca de tecnologia ao longo do tempo. Se a aplicação migrasse de um banco relacional para outro mecanismo de armazenamento, ou se ganhasse uma nova interface pública via HTTP, o núcleo de negócio permaneceria essencialmente o mesmo. A principal contribuição prática do padrão, nesse cenário, seria manter a lógica central estável enquanto a borda do sistema evolui conforme as necessidades do produto.

4. Conclusão

A principal contribuição do artigo é deslocar o foco arquitetural da separação superficial em camadas para uma separação mais profunda entre núcleo de negócio e mundo externo. Com isso, Cockburn mostra que portas e adaptadores não são apenas uma forma elegante de organizar código, mas um meio concreto de elevar testabilidade, reduzir acoplamento e aumentar a capacidade de evolução do sistema. O texto também é valioso porque evita simplificações excessivas: ele reconhece que a arquitetura exige critério na definição das portas e disciplina para não reintroduzir a complexidade nos lugares errados. Como aprendizado central, fica a ideia de que uma boa arquitetura não isola apenas tecnologias; ela protege a intenção do software.